C PROGRAMMING - DEBUGGING TOOLS

Practical Assignment Report

Name: Dhruv Pandey

SAP ID: 590037660

Course: B.Tech CSE - 1st Year

Batch: 15

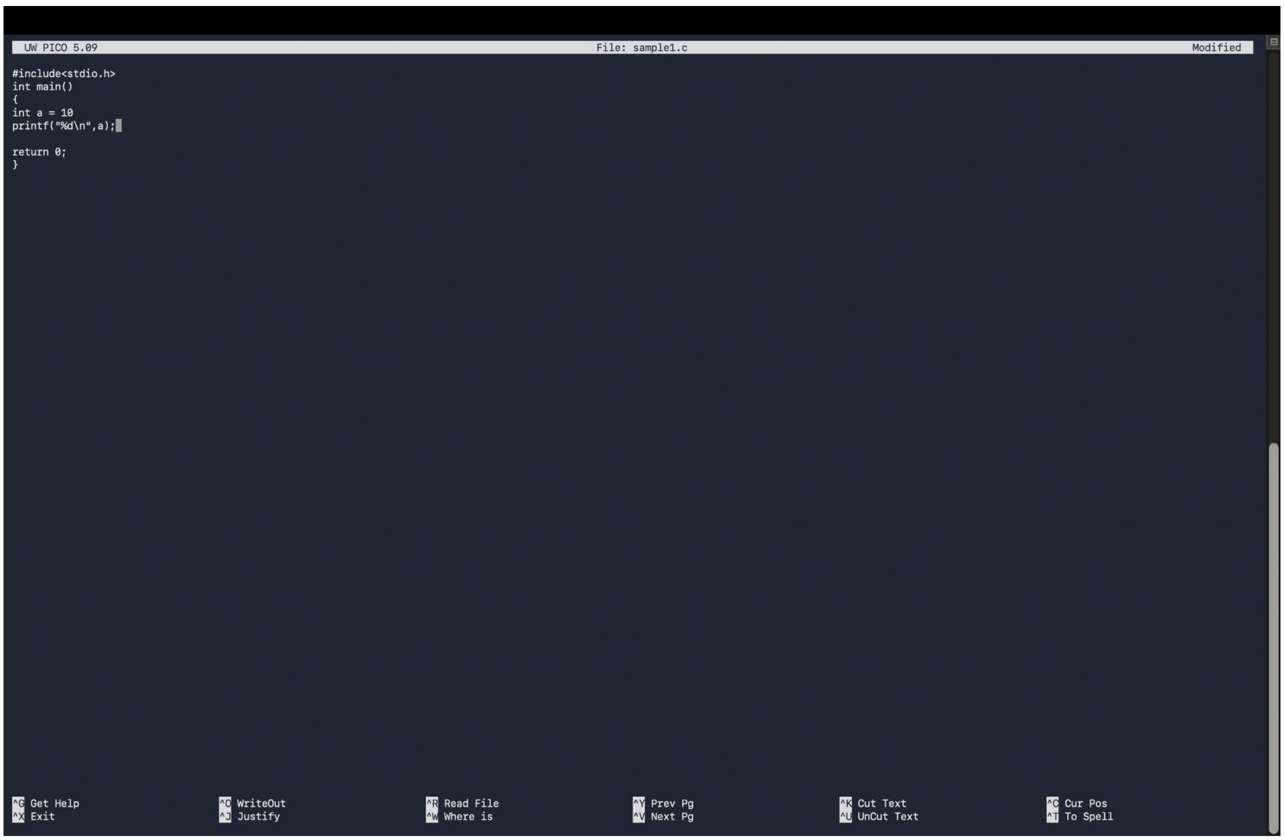
1. Fundamentals of Errors and Debugging

An error is a mistake in source code. A bug is a defect that causes incorrect or unexpected behavior. Debugging is the systematic process of finding, analyzing, and removing such defects.

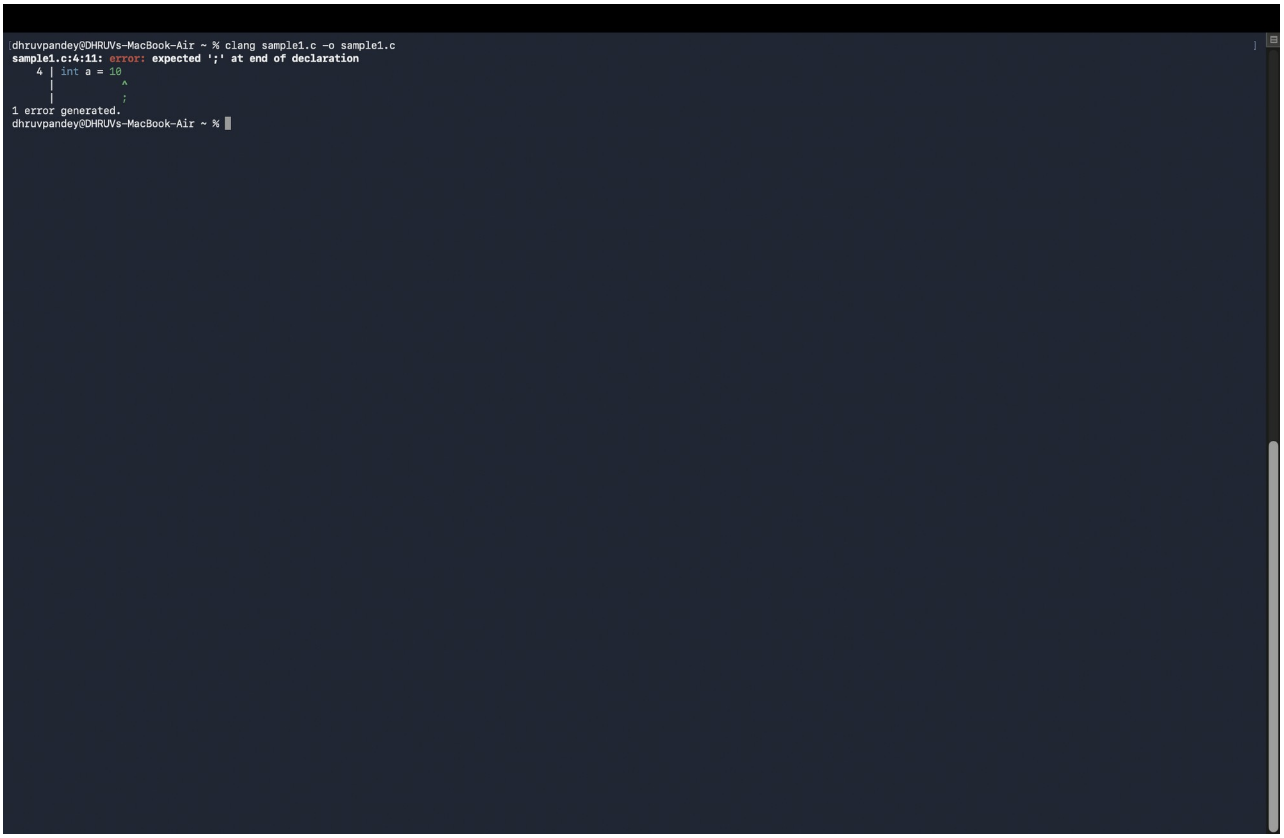
Error Type	Meaning	Typical Example
Syntax	Violates C grammar	Missing semicolon
Compilation/Type	Invalid type or declaration	Incompatible assignment
Linker	Required definition cannot be linked	Undefined function
Runtime	Fails during execution	Division by zero / invalid memory
Logical	Runs but gives wrong result	Incorrect loop condition

Case 1: Syntax & Compilation Error

A missing semicolon prevents successful compilation. Clang identifies the source line and reports that a semicolon is expected.



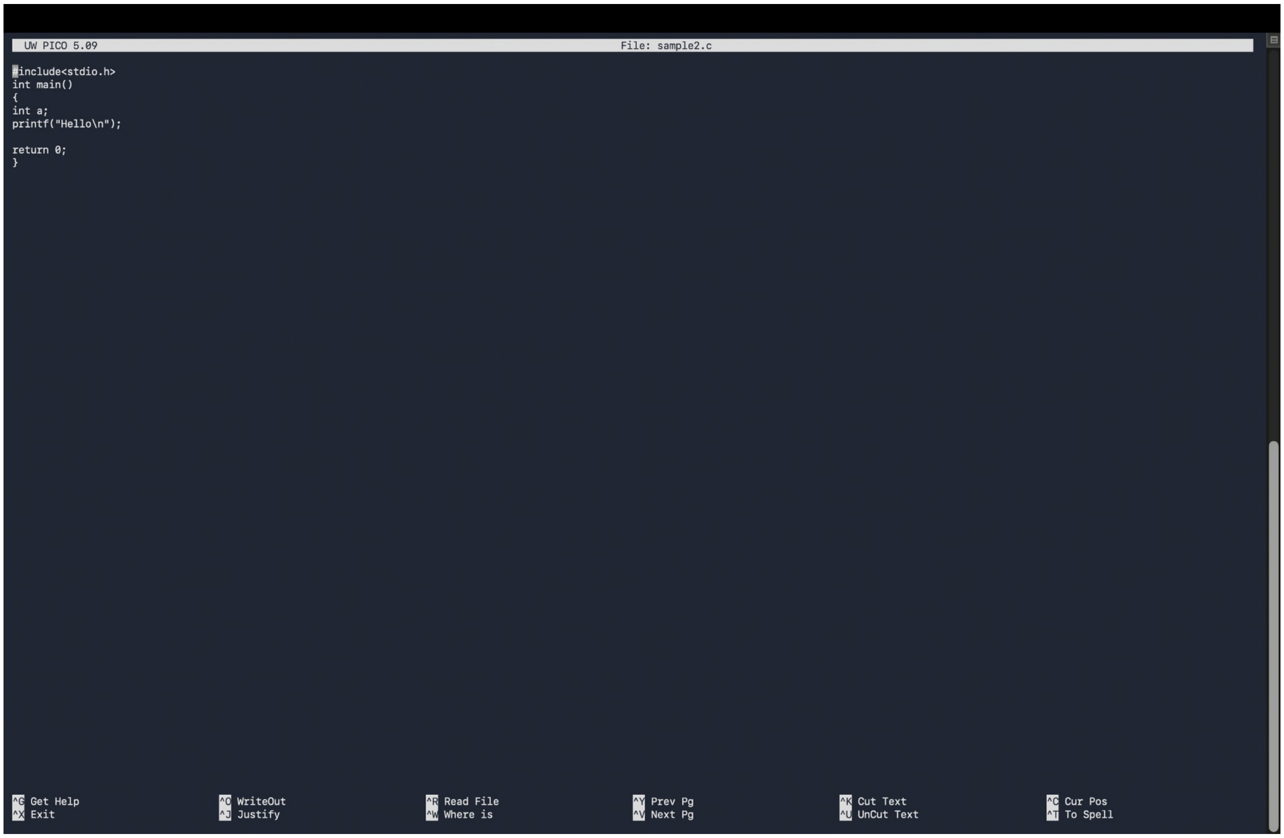
Source code containing the syntax error.



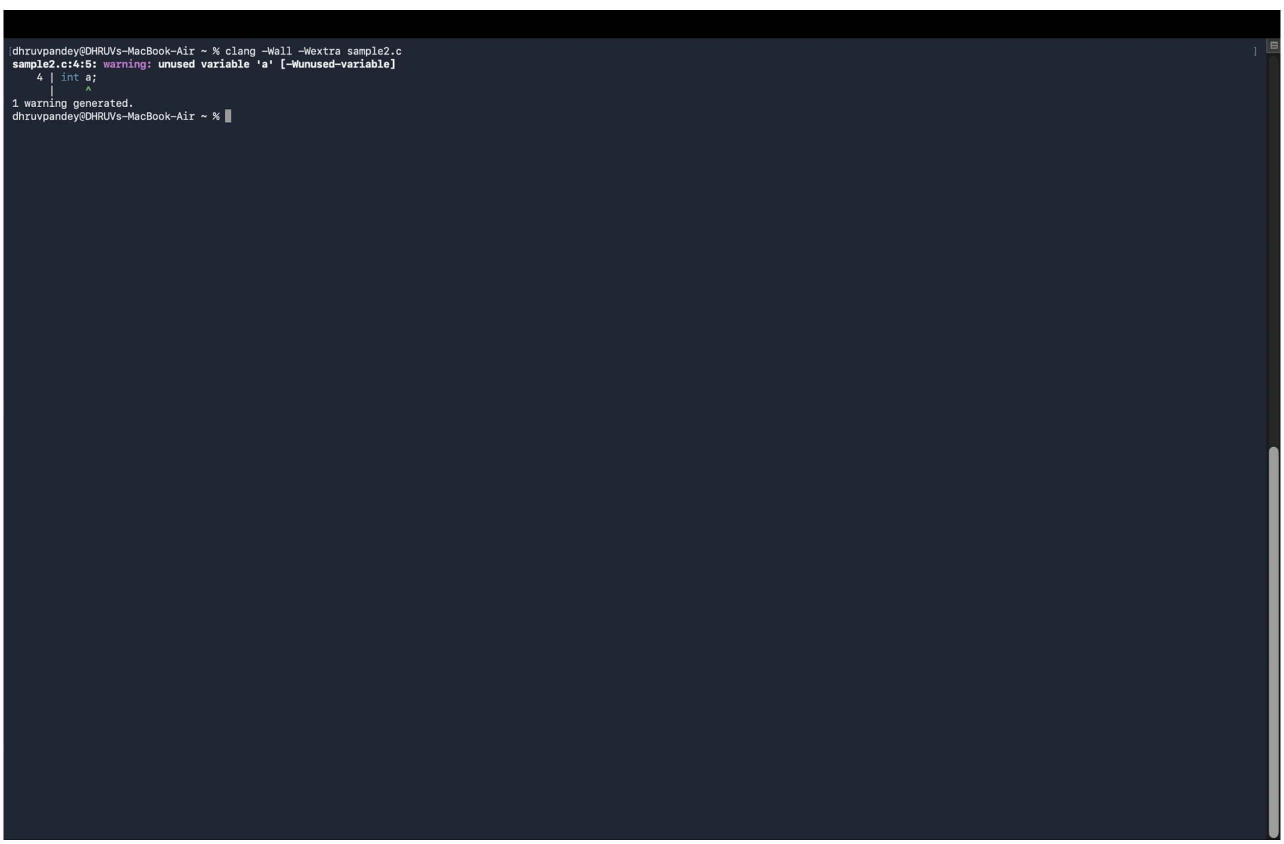
Terminal output showing the Clang syntax-error diagnostic.

Case 2: Compiler Warnings (-Wall, -Wextra)

Warnings identify suspicious code that is still legal C. Here, variable a is declared but never used.



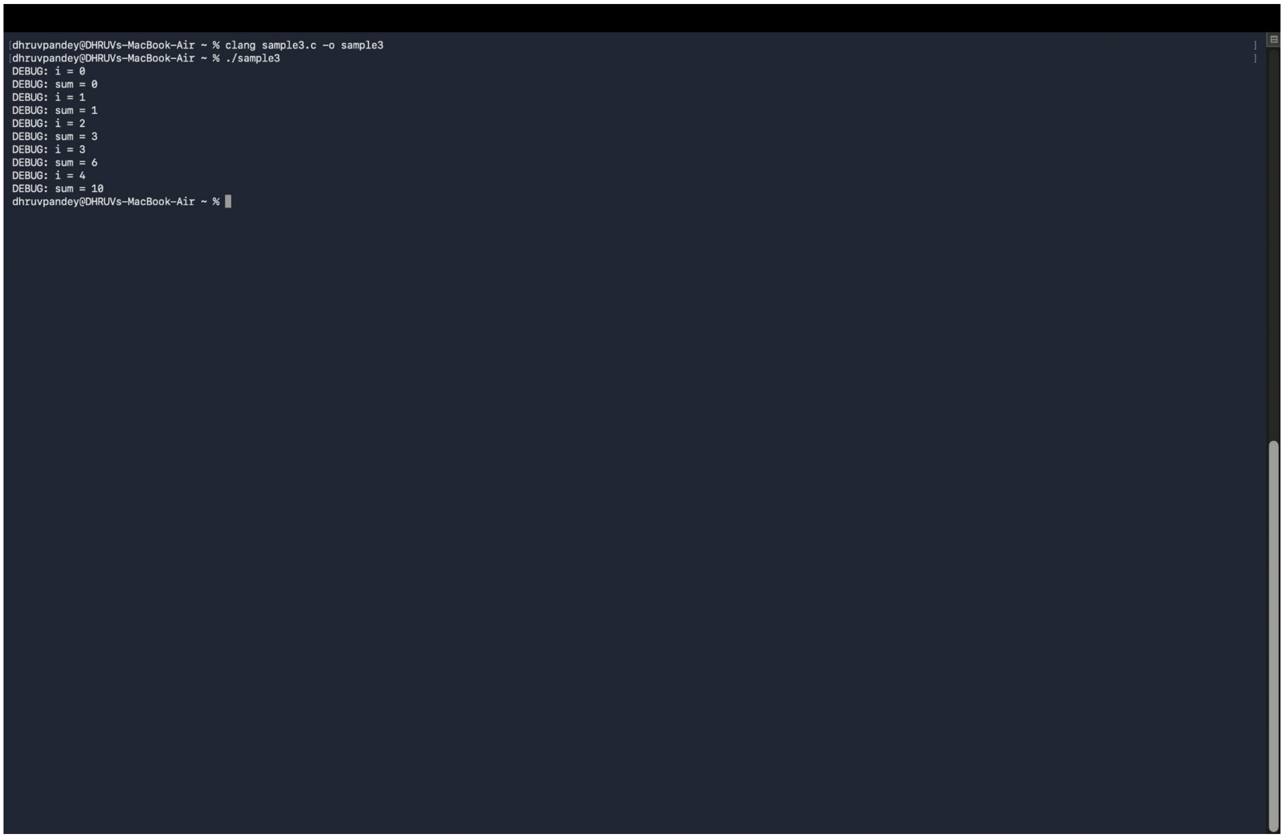
Program containing an unused variable.



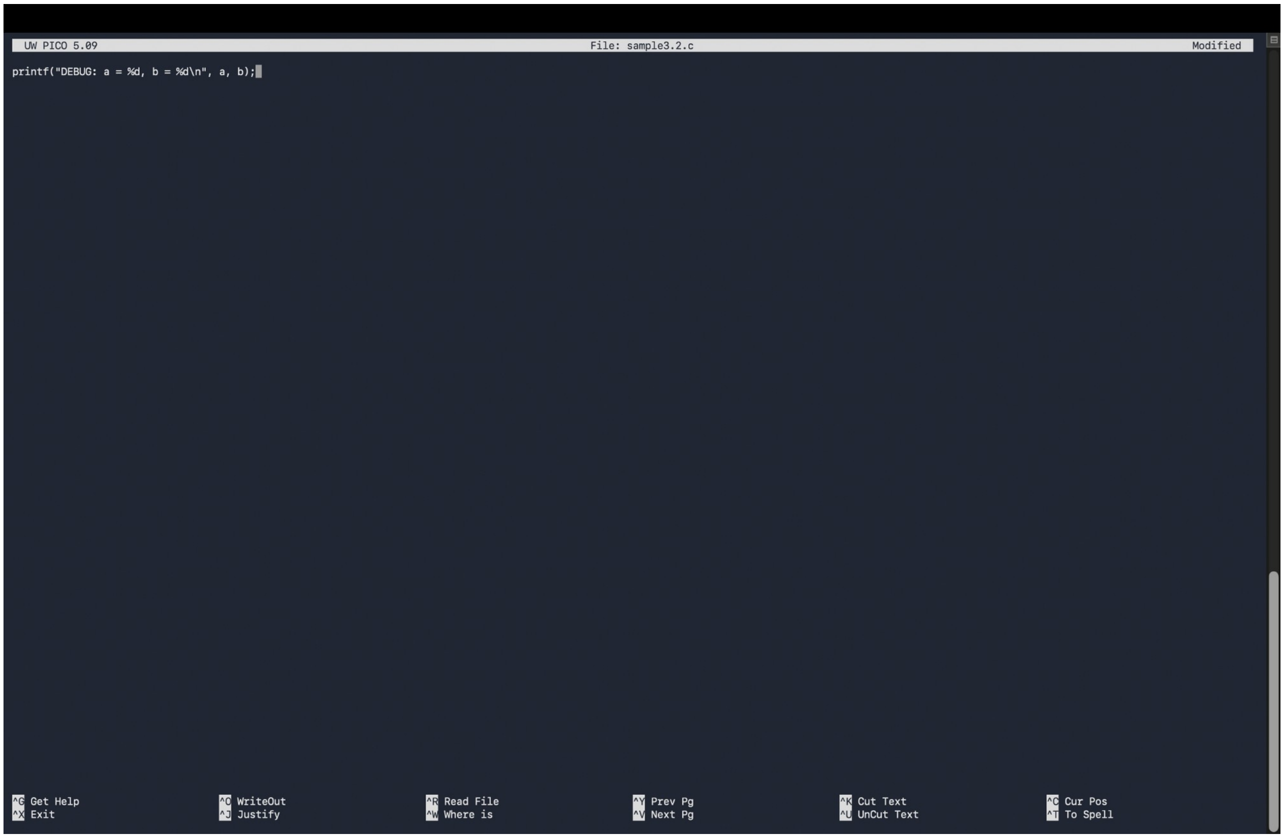
Terminal warning generated with -Wall and -Wextra.

Case 3: printf-Based Debugging (Logical Bug)

Debug print statements expose the values of i and sum on each loop iteration. The trace shows the program reaches a final sum of 10.



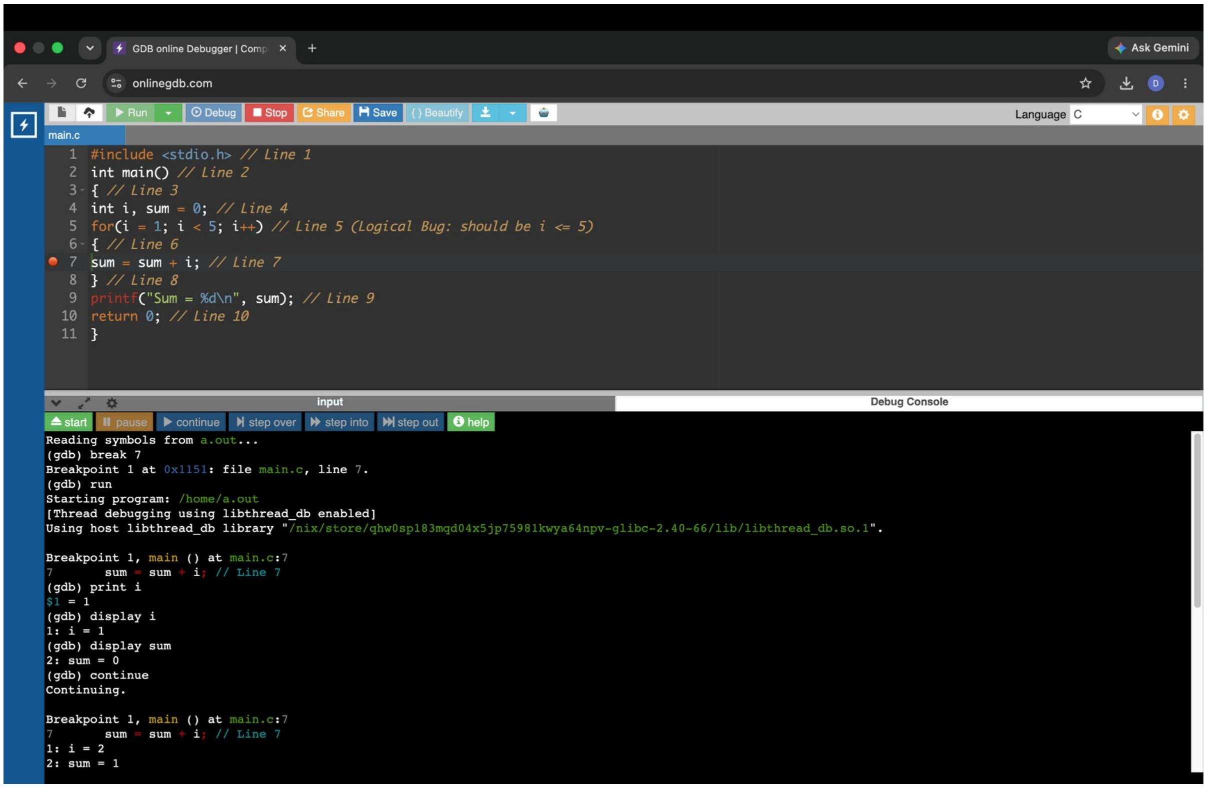
Terminal trace produced by printf-based debugging.

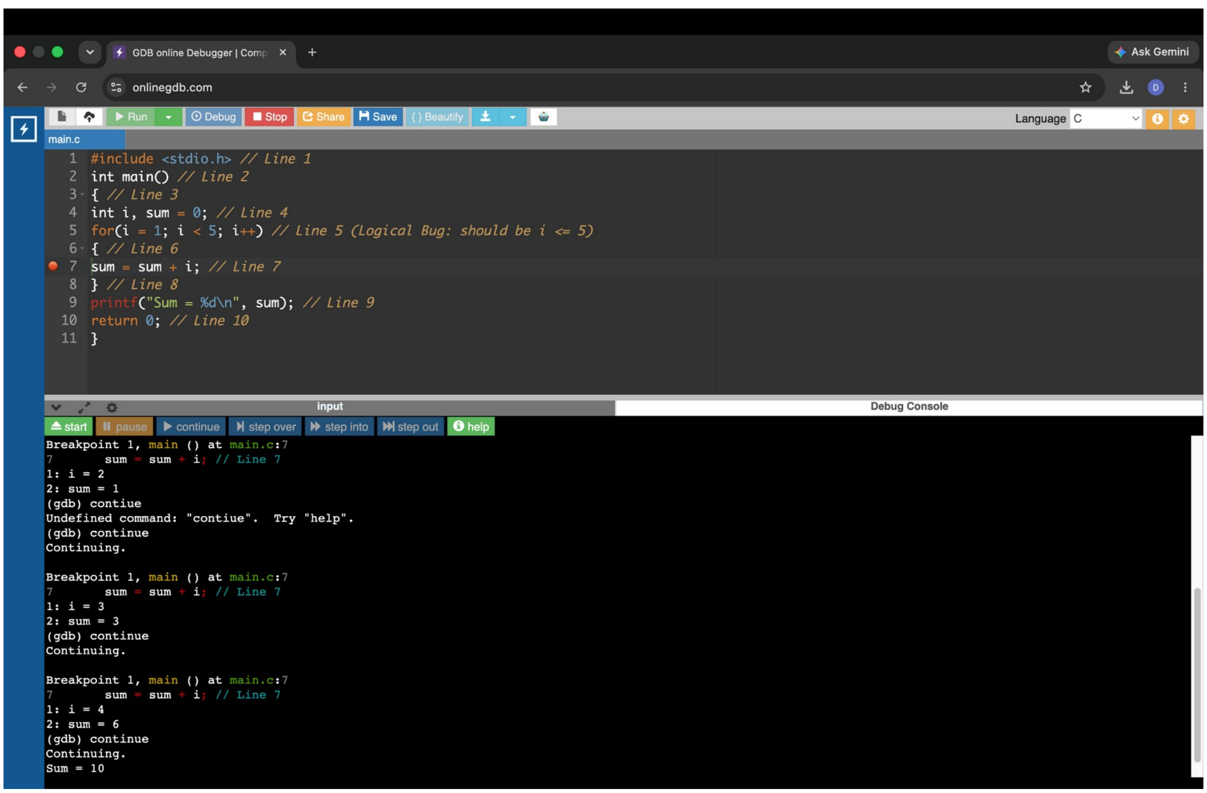


Example of a debug printf statement.

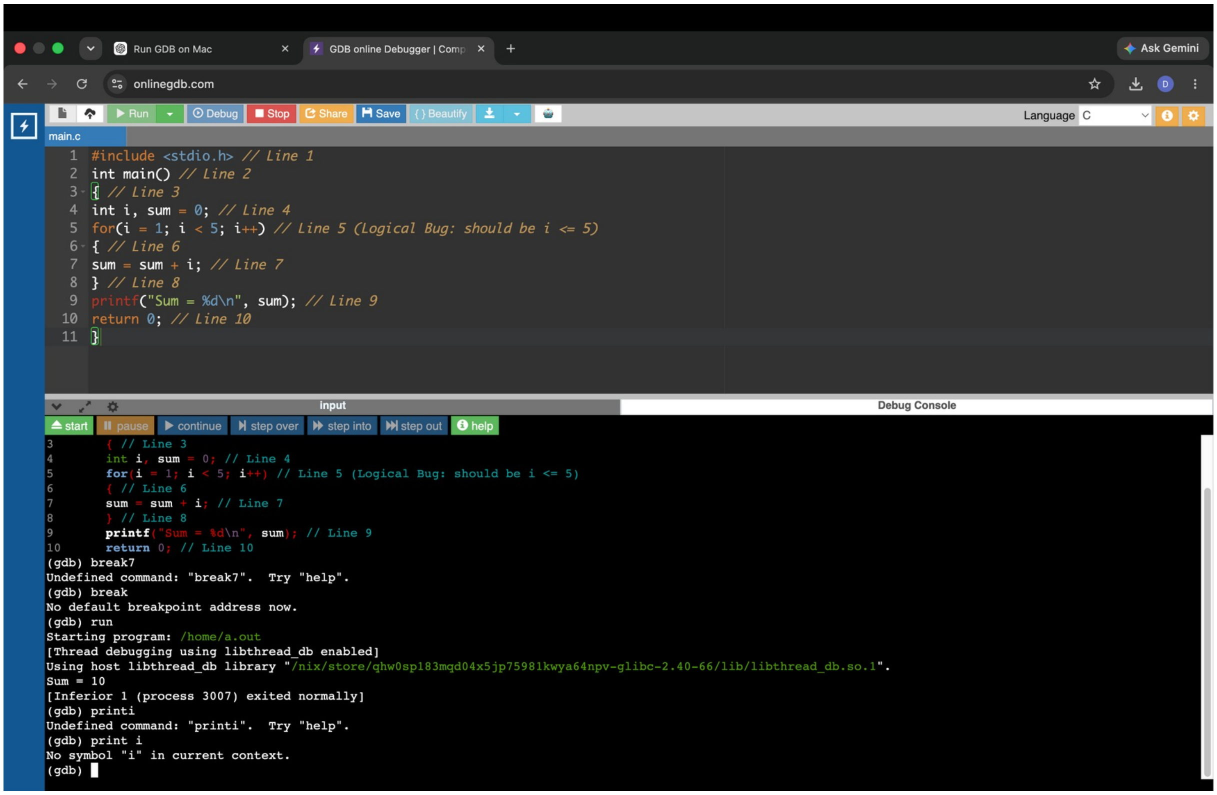
Case 4: Debugging a Logical Error using GDB

The loop uses i < 5, so 5 is never included. GDB can stop on the sum statement and display i and sum at each iteration. The actual result is 10; the intended result is 15.





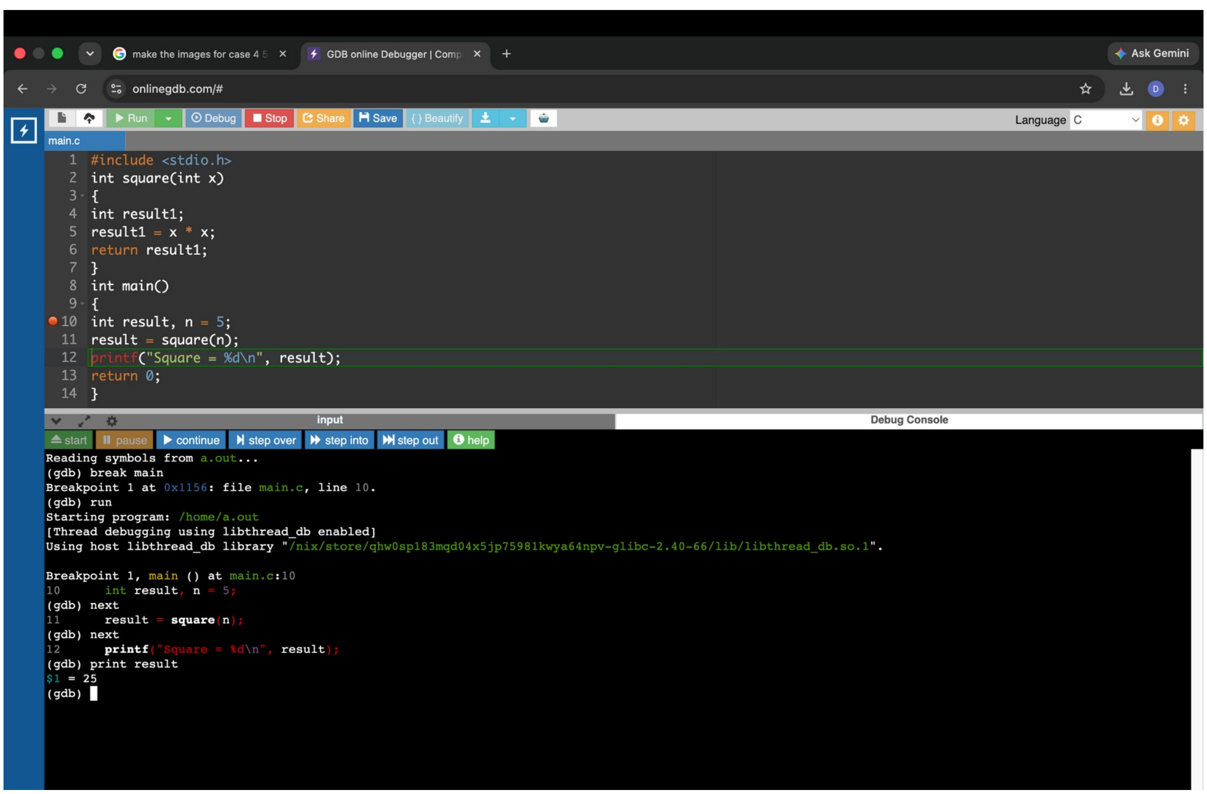
GDB observation: when i reaches 5, the loop condition becomes false before the body executes. Therefore 5 is omitted from the sum.

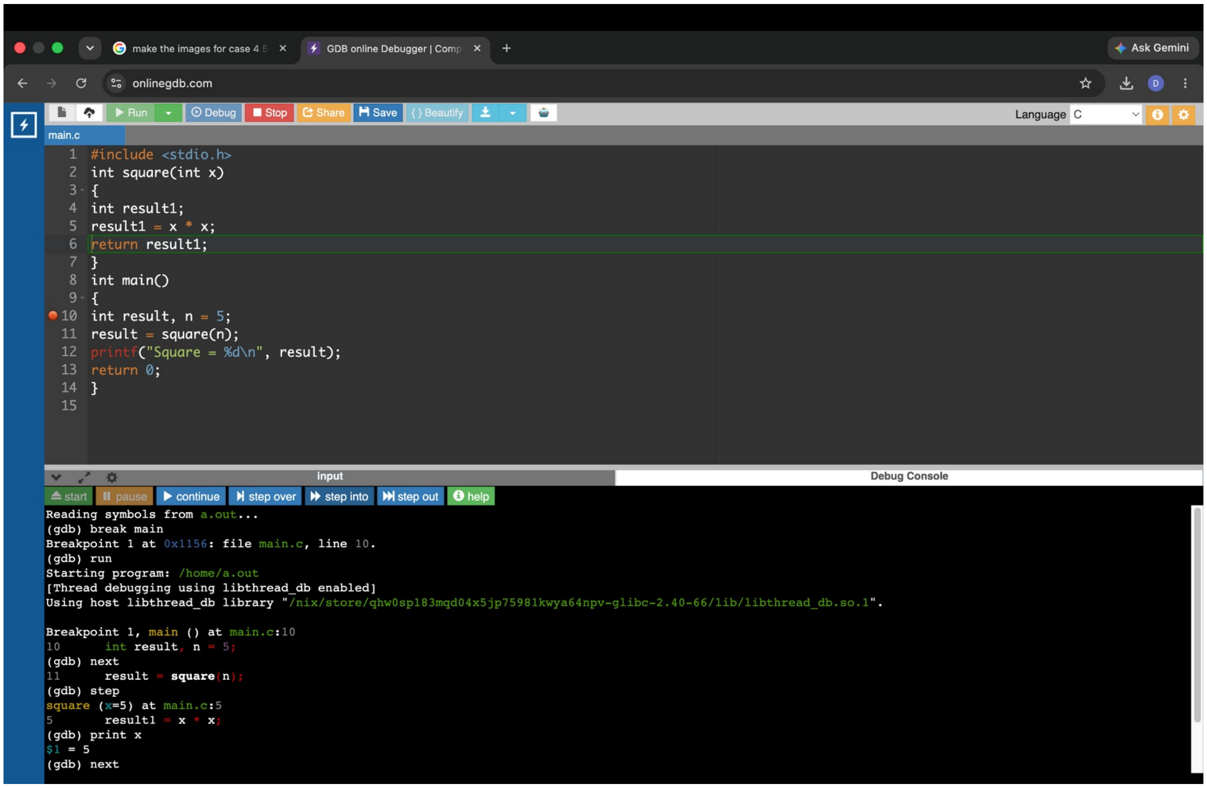


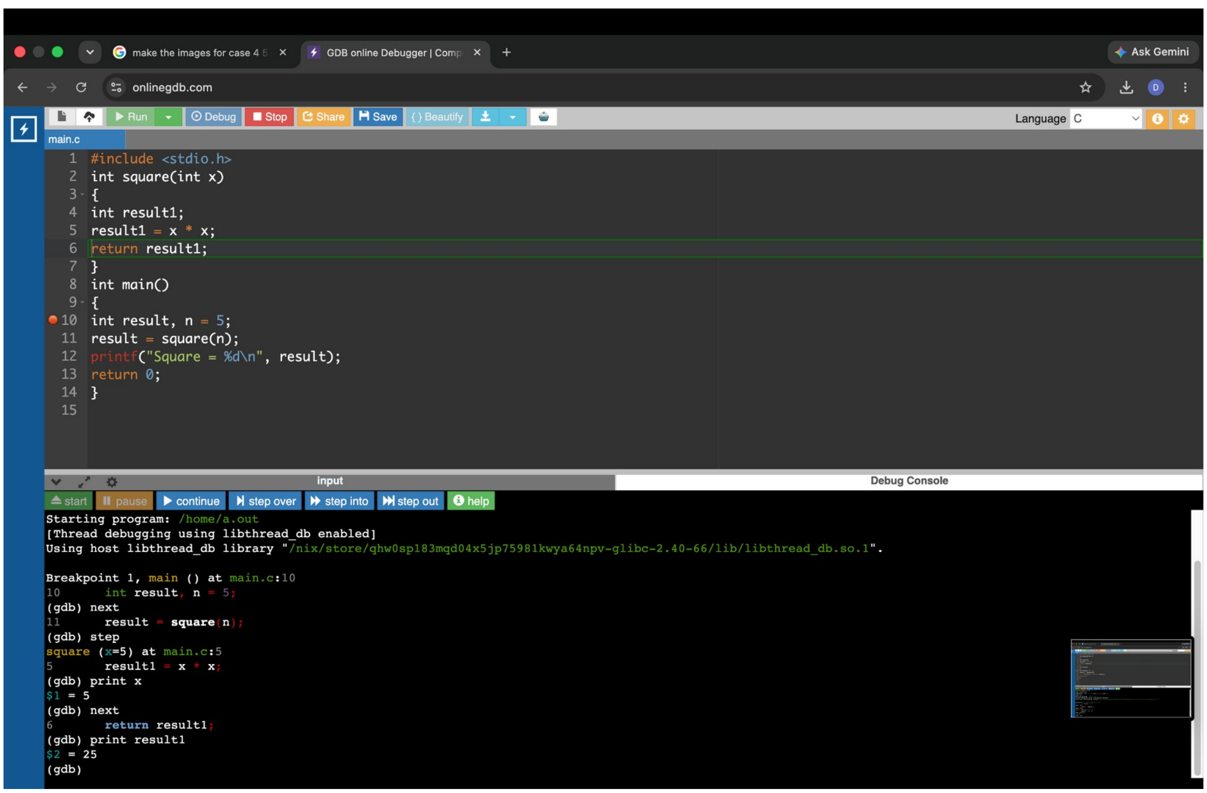
OnlineGDB logical-error debugging session.

Case 5: Debugging Functions (next vs. step)

GDB next executes a function call without entering it, while step enters the called function so its statements and local values can be inspected.

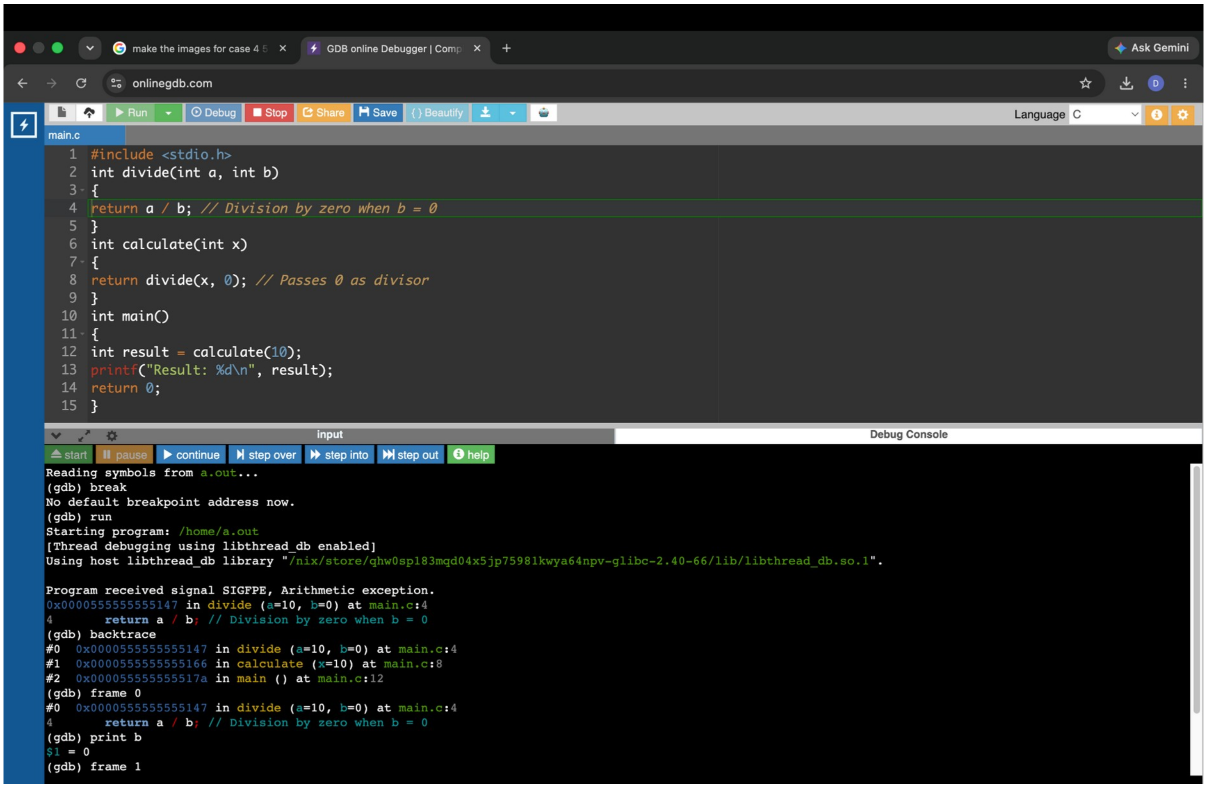


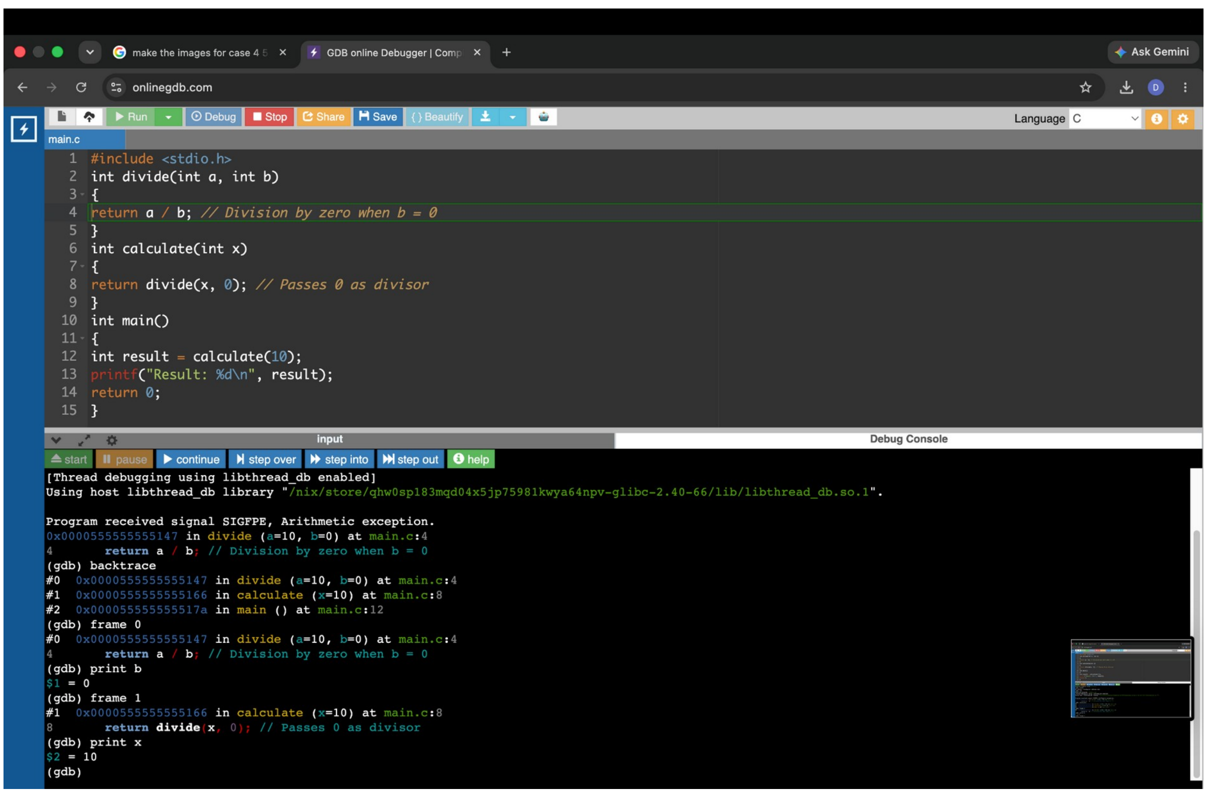




Case 6: Runtime Error using GDB Backtrace

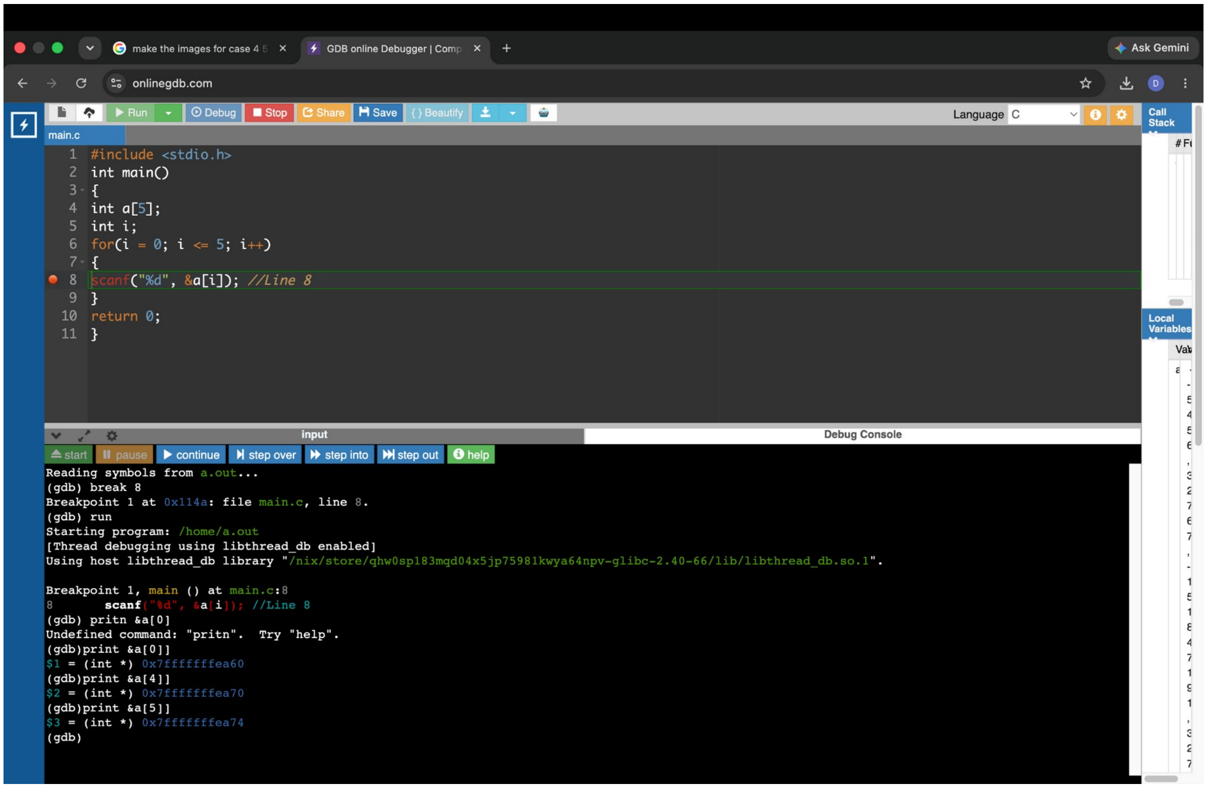
A division by zero produces SIGFPE. The backtrace command shows the chain divide -> calculate -> main, making it possible to locate the origin of the crash.





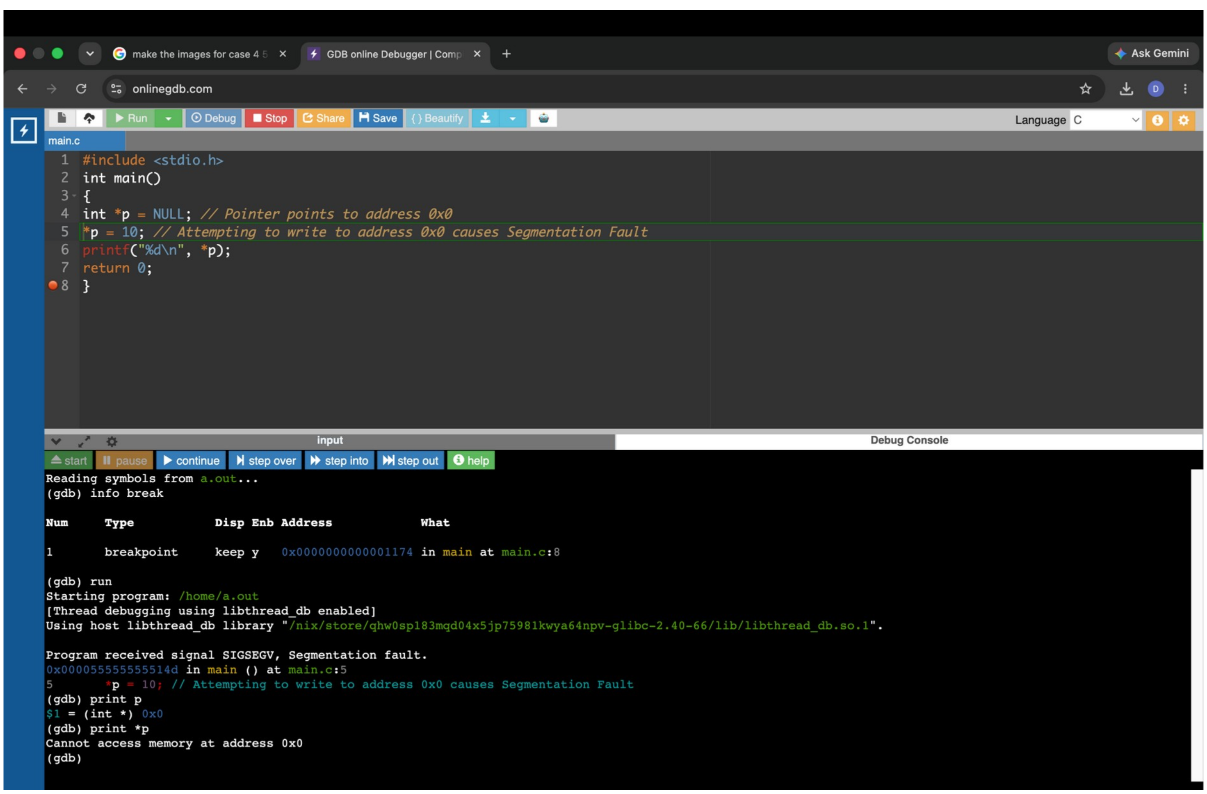
Case 7: Array Out-of-Bounds Error using GDB

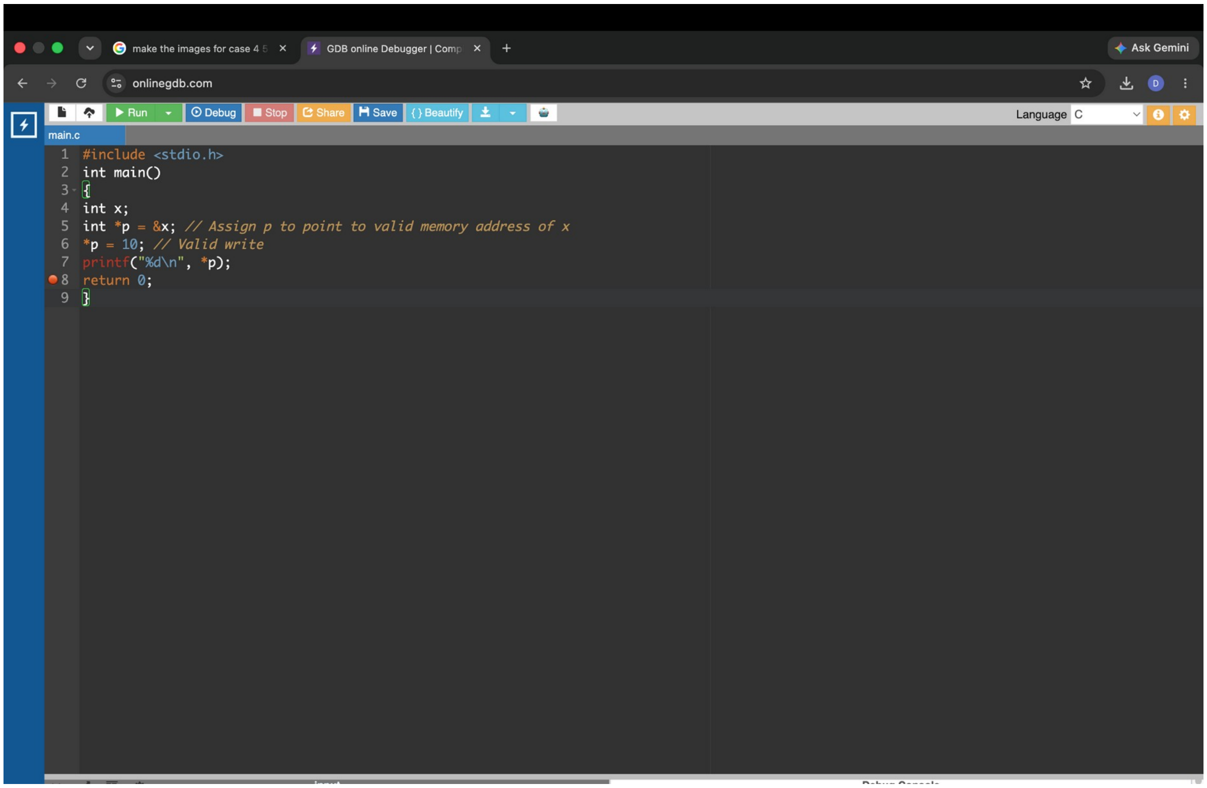
An array of five integers has valid indices 0 through 4. The condition i <= 5 also accesses a[5], which is outside the array and causes undefined behavior.



Case 8: Segmentation Fault (SIGSEGV) using GDB

The pointer p is NULL (address 0x0). Writing through it with *p = 10 attempts to access invalid memory, so the program receives SIGSEGV.





Systematic Debugging Strategy

1. Observe expected vs. actual output
2. Reproduce the issue consistently
3. Compile with -Wall -Wextra -g
4. Isolate suspicious code
5. Inspect state using printf or GDB
6. Apply one minimal fix
7. Recompile and retest
8. Verify edge cases and boundary inputs

Conclusion

Compiler diagnostics, warning flags, printf tracing, and GDB provide complementary ways to locate errors. Breakpoints and variable inspection help with logical bugs; next and step help analyze functions; backtrace identifies crash call chains; and pointer/array inspection helps identify invalid memory access.